

Software Requirements Specification

for

TRIPWISE

Prepared by

23DCE066, 23DCE076 and 23DCE077

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Project Scope	1
1.5 References.....	1
2. Overall Description	2
2.1 Product Perspective.....	2
2.2 Product Features	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment.....	2
2.5 Design and Implementation Constraints	2
2.6 User Documentation	3
2.7 Assumptions and Dependencies	3
3. System Features	3
3.1 System Feature 1.....	3
3.2 System Feature 2 (and so on).....	Error! Bookmark not defined.
4. External Interface Requirements	5
4.1 User Interfaces	5
4.2 Hardware Interfaces	5
4.3 Software Interfaces	5
4.4 Communications Interfaces	6
5. Other Nonfunctional Requirements.....	6
5.1 Performance Requirements	6
5.2 Safety Requirements	6
5.3 Security Requirements	6
5.4 Software Quality Attributes	7
6. Other Requirements	7
Appendix A: Glossary.....	7
Appendix B: Analysis Models	7
Appendix C: Issues List.....	7

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

TripWise is an intelligent travel planner designed to help users create, personalize, and optimize travel itineraries. AI Trip Planner is a web-based application built using the MERN stack to provide personalized travel itineraries. It leverages AI to suggest destinations, accommodations, and activities based on user preferences while integrating real-time data like weather, flights, and budget estimates. This SRS outlines the system's key functionalities, constraints, and requirements for development.

1.2 Document Conventions

This SRS follows standard formatting for clarity and consistency. Key conventions include:

- Bold headings for emphasis.*
- Requirements hierarchy* where high-level needs inherit detailed specifications.*
- Terminology: "User" for travelers, "Admin" for system operators.*
- Date format: YYYY-MM-DD.*

1.3 Intended Audience and Reading Suggestions

This document is intended for:

SGP Team Members: The three students developing the project.

Project Guide and Review Faculty: To provide guidance and feedback on the project.

1.4 Project Scope

TripWise aims to simplify travel planning by integrating AI-driven suggestions, itinerary creation, and budget tracking. The key features include:

AI-Powered Place Recommendations

Automated Itinerary Generator

Budget Tracker

User Profile and Saved Trips

1.5 References

This SRS may refer to related documents such as UI/UX guidelines, API documentation, system architecture diagrams, and external travel data sources.

2. Overall Description

2.1 Product Perspective

TripWise is a **new, self-contained** product designed to assist travelers in planning their trips efficiently. It does not replace an existing system but is built as an independent solution to enhance the travel planning experience. The system will interact with external APIs to fetch travel data and provide AI-driven recommendations. Below is a high-level overview of the system:

- **Frontend:** React.js-based user interface
- **Backend:** Node.js with Express.js
- **Database:** MongoDB
- **External APIs:** Integration with travel and mapping services

2.2 Product Features

The key features of TripWise include:

- **AI-Powered Place Recommendations:** Suggests travel destinations based on user preferences.
- **Automated Itinerary Generator:** Creates optimized trip plans.
- **Budget Tracker:** Helps users manage travel expenses.
- **User Profiles:** Allows users to save trips and track past itineraries.

2.3 User Classes and Characteristics

TripWise caters to the following user categories:

- **Casual Travelers:** Occasional users planning leisure trips.
- **Frequent Travelers:** Users who travel regularly for business or leisure.
- **Travel Enthusiasts:** Individuals who explore different destinations frequently.
- **Tour Operators:** Businesses that may use the tool to plan itineraries for clients.

2.4 Operating Environment

Supported Platforms: Web browsers (Chrome, Firefox, Safari, Edge)

Operating System Compatibility: Windows, macOS, Linux

Internet Connectivity: Required for real-time updates.

2.5 Design and Implementation Constraints

Technology Stack: React.js, Node.js, MongoDB

APIs: Dependency on third-party travel and mapping APIs

Security: User authentication and data encryption

Performance: Optimization for fast itinerary generation

2.6 User Documentation

The following documentation will be provided:

User Manual: Detailed guide on how to use TripWise

Online Help: FAQs and troubleshooting tips

Developer API Documentation: For integrating with external systems

2.7 Assumptions and Dependencies

Internet access is required for fetching live travel data.

The system relies on third-party APIs for location-based recommendations.

Users must create an account to save and manage itineraries.

Performance may depend on the efficiency of external API responses.

3. System Features

3.1 AI-Powered Place Recommendations

3.1.1 Description and Priority

This feature provides users with AI-driven travel destination suggestions based on their preferences, budget, and season.

Priority: High

Benefit: Enhances user experience by providing intelligent recommendations.

Penalty: Users may find manual planning time-consuming without AI support.

Cost: Medium (depends on API usage and AI model optimization).

Risk: Dependency on external APIs for fetching travel data.

3.1.2 Stimulus/Response Sequences

User inputs preferences (e.g., budget, interests, duration).

System fetches relevant destination data from APIs.

System presents a list of recommended destinations.

3.1.3 Functional Requirements

REQ-1: The system shall allow users to input travel preferences.

REQ-2: The system shall process user data and generate recommendations.

REQ-3: The system shall display recommendations with key travel details.

REQ-4: The system shall allow users to save recommended places.

3.2 Automated Itinerary Generator

3.1.1 Description and Priority

This feature creates optimized travel plans based on user-selected destinations and constraints.

Priority: High

Benefit: Saves users time by automating itinerary creation.

Penalty: Users must manually plan their trips if this feature is absent.

Cost: High (AI-based optimization requires computation resources).

Risk: Complex itineraries may require multiple iterations for accuracy.

3.1.2 Stimulus/Response Sequences

User selects destinations and trip duration.

System fetches data on locations, transport, and accommodation.

System optimizes a schedule based on travel feasibility.

System presents a suggested itinerary with a timeline.

3.1.3 Functional Requirements

REQ-1: The system shall allow users to input trip details.

REQ-2: The system shall generate an optimized itinerary.

REQ-3: The system shall display travel times and estimated costs.

REQ-4: The system shall allow users to modify the itinerary manually.

3.3 Budget Tracker

3.1.1 Description and Priority

This feature helps users manage their travel expenses efficiently.

Priority: Medium

Benefit: Ensures users stay within their travel budget.

Penalty: Users may overspend without real-time tracking.

Cost: Low (simple calculations and UI components).

Risk: Accuracy depends on real-time exchange rate and cost updates.

3.1.2 Stimulus/Response Sequences

User inputs estimated trip expenses.

System categorizes and tracks expenses.

System provides real-time budget updates.

3.1.3 Functional Requirements

REQ-1: The system shall allow users to enter a travel budget.

REQ-2: The system shall track expenses in various categories.

REQ-3: The system shall display real-time spending updates.

REQ-4: The system shall alert users if they exceed their budget.

4. External Interface Requirements

4.1 User Interfaces

The system will feature a responsive and intuitive user interface designed with modern UI/UX principles. Key aspects include:

GUI Standards: Consistent layout with navigation bar and user-friendly design.

Common Elements: Standard buttons like Home, Save, Edit, and Delete.

Error Handling: Clear error messages with troubleshooting suggestions.

Accessibility: Compatible with screen readers and keyboard navigation.

4.2 Hardware Interfaces

TripWise is a web-based application and does not require specialized hardware. However, it supports:

Device Compatibility: Desktop, laptop, tablet, and mobile devices.

Internet Connectivity: Required for fetching travel data and processing requests.

Storage Requirements: Minimal local storage; cloud-based data management.

4.3 Software Interfaces

TripWise integrates with various external software systems, including:

Databases: MongoDB for user data storage.

Operating Systems: Windows, macOS, and Linux (via web browsers).

Third-party APIs: Integration with travel services for recommendations and itinerary planning.

Authentication: Secure login with OAuth 2.0 and JWT-based authentication.

4.4 Communications Interfaces

The system requires internet access for seamless communication and data exchange. Supported communication protocols include:

HTTPS: Secure data transmission between clients and servers.

REST APIs: Interaction with third-party travel and booking services.

Email Notifications: User alerts and itinerary updates.

Data Encryption: Ensuring user privacy and security through SSL/TLS encryption.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Response Time: The system must process user requests within 2 seconds during peak load times (500 users).

Throughput: The system must handle up to 1000 concurrent users without significant degradation in response time or performance.

Latency: For real-time communication features (e.g., AI suggestions, itinerary generation), response time must not exceed 1 second.

Scalability: The system should be scalable to handle increased load during peak seasons without requiring significant redesign or manual intervention.

Rationale: These performance targets are set to ensure a seamless user experience, particularly during high traffic periods such as holiday seasons. Meeting these performance targets ensures the platform remains responsive and reliable under stress.

5.2 Safety Requirements

Data Protection: Sensitive user data, such as travel plans and payment details, must be encrypted both in transit and at rest using industry-standard encryption techniques.

Error Handling: In case of system failure, an automated recovery process must be in place to prevent data corruption or loss, with clear messaging to the user.

Access Control: Users should only be able to access data they have permission to view, with system access divided by roles and responsibilities.

5.3 Security Requirements

User Authentication: All users must be authenticated via a secure login system, using OAuth or other secure protocols.

Data Privacy: The product must comply with applicable data protection regulations, including GDPR, by implementing necessary measures such as user consent, data anonymization, and secure data handling practices.

External Security Audits: The system should undergo regular security audits to ensure that security vulnerabilities are identified and mitigated promptly.

Multi-Factor Authentication (MFA): Optional for users accessing sensitive data or performing high-risk actions (e.g., payment processing).

5.4 Software Quality Attributes

Usability: The user interface must be intuitive and easy to navigate, with a clear design that facilitates quick decision-making. The system must undergo usability testing with a goal of at least 80% user satisfaction on the first use.

Reliability: The system should have 99.9% uptime and be capable of recovering from failures without losing user data.

Maintainability: The codebase must be modular, well-documented, and follow industry best practices to ensure ease of maintenance and feature expansion.

Interoperability: The system must work seamlessly with third-party platforms such as Google Maps, payment gateways, and social media integrations.

Portability: The system must be adaptable to mobile devices, particularly for Android and iOS platforms.

6. Other Requirements

Appendix A: Glossary

This section defines all the terms, acronyms, and abbreviations necessary to interpret the SRS, ensuring clarity for all stakeholders. Examples include:

API (Application Programming Interface): A set of tools and protocols that allow different software components to communicate.

UI (User Interface): The visual elements that allow users to interact with the system.

GDPR (General Data Protection Regulation): A regulation in EU law that focuses on data protection and privacy for individuals.

Appendix B: Analysis Models

Data Flow Diagram (DFD): A visual representation of data processes and their interactions within the system.

Entity-Relationship Diagram (ERD): A diagram that shows the relationships between various entities in the system (e.g., users, trips, payments).

Class Diagram: A visual representation of the system's classes and their relationships, useful for object-oriented design.

State-Transition Diagram: A diagram illustrating the possible states of a system or component and the events that cause transitions between these states.

Appendix C: Issues List

BD: Finalize API structure for third-party integration (Waiting for feedback from the partner team).

Pending Decision: Choice of payment gateway to use for international transactions (Options: Stripe vs. PayPal).

Conflict Resolution: The current UI design does not meet accessibility guidelines; further discussions required to adjust layout for screen readers.